

EPISODIC BAYESIAN RACING: BRIDGING THE GAP BETWEEN EFFICIENCY AND ADAPTABILITY IN HYPERPARAMETER OPTIMIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Modern hyperparameter optimization (HPO) methods present a challenging trade-off between computational overhead and adaptive decision-making. Low-overhead schedulers like ASHA and BOHB are computationally cheap but rely on rigid, pre-defined promotion schedules, limiting their adaptability. Conversely, fully dynamic methods such as DyHPO offer fine-grained, per-evaluation control by constantly refitting a surrogate model, but this incurs prohibitive computational costs that can dominate the optimization process. This creates a critical gap for a method that is both highly adaptive and computationally efficient. We introduce CAR-HPO (Checkpoint-based Adaptive Racing for HPO), a novel episodic Bayesian racing algorithm designed to occupy this sweet spot. Our core contribution is an amortized intelligence mechanism where decisions are triggered only at adaptive ‘checkpoints’. At each checkpoint, we refit a single, powerful multi-output Gaussian Process model to the entire performance history. This unified model is then used for two concurrent, model-driven decisions: risk-averse predictive promotion of promising configurations and cost-aware adaptive sampling of new ones. By amortizing the cost of model fitting over batches of evaluations, CAR-HPO makes robust, intelligent decisions without the crippling overhead of fully dynamic methods. We validate our approach on tabular benchmarks from HPOBench, demonstrating that CAR-HPO achieves sample efficiency competitive with or superior to the state-of-the-art DyHPO, while reducing scheduler overhead by a factor of five. This establishes CAR-HPO as a powerful, scalable, and practical tool that significantly optimizes the performance-per-computation trade-off in HPO.

1 INTRODUCTION

The performance of modern machine learning models is critically dependent on the careful selection of hyperparameters Elsken et al. (2018); Mlodozieniec et al. (2023). Hyperparameter optimization (HPO) has thus become an indispensable step in the machine learning pipeline. Multi-fidelity optimization techniques have emerged as a powerful paradigm for accelerating HPO, leveraging cheap, low-fidelity approximations of model performance (e.g., training for fewer epochs) to quickly prune unpromising configurations Li et al. (2020). However, existing multi-fidelity methods exist on a spectrum of overhead versus adaptability.

On one end, schedulers like Asynchronous Successive Halving (ASHA) offer extremely low overhead but employ rigid, heuristic-based rules for promoting configurations to higher budgets. Methods like BOHB enhance this by incorporating a model-based sampler, but promotions remain heuristic. This static nature can lead to suboptimal decisions, especially in complex search spaces where performance at low budgets is not perfectly indicative of final performance.

On the other end of the spectrum lie fully dynamic, adaptive methods like DyHPO Wistuba et al. (2022). These methods refit a probabilistic surrogate model after every single trial evaluation, enabling highly granular and responsive decisions about which configuration to evaluate next and for how long. While this approach can yield state-of-the-art sample efficiency, it comes at a steep price: the computational cost of constant model refitting can become prohibitive, often consuming a significant portion of the total optimization budget. This high overhead makes such methods impractical for many

real-world scenarios, particularly when tuning already expensive models where every computational cycle counts.

This landscape reveals a critical gap for an HPO method that combines the adaptive intelligence of model-based dynamic methods with the computational efficiency of simpler static schedulers. We propose to fill this gap with CAR-HPO (Checkpoint-based Adaptive Racing for HPO), a novel algorithm based on the principle of 'Episodic Bayesian Racing'. Instead of reacting to every new data point, CAR-HPO amortizes its intelligence over batches of evaluations. It operates asynchronously but triggers a centralized decision-making process only at adaptive 'checkpoints'. At these moments, it fits a single, unified multi-output Gaussian Process (GP) model to the entire history of observations. This powerful, up-to-date model is then used to make two principled, concurrent decisions: which existing configurations to promote based on a predictive, risk-averse criterion, and which new configurations to sample using a cost-aware acquisition function. By decoupling the frequency of model updates from the frequency of trial evaluations, CAR-HPO drastically reduces computational overhead while still making robust, model-informed decisions.

To validate our approach, we conduct a rigorous experimental comparison on tabular HPO benchmarks, which allow for the precise measurement of scheduler overhead. Our contributions are as follows:

- We introduce CAR-HPO, a novel episodic Bayesian racing algorithm that effectively balances the trade-off between adaptability and computational efficiency in multi-fidelity HPO.
- We propose an amortized model-fitting strategy triggered at adaptive checkpoints, which significantly reduces the computational overhead compared to fully dynamic methods.
- We develop a unified, model-driven decision-making framework where a single multi-output Gaussian Process model is used for both principled predictive promotion and adaptive infill sampling.
- We empirically demonstrate that CAR-HPO achieves sample efficiency competitive with the state-of-the-art, high-overhead DyHPO method, while maintaining a computational cost comparable to simpler schedulers like BOHB.

2 RELATED WORK

Our work builds upon the rich literature of multi-fidelity hyperparameter optimization. The foundation of many modern methods is Successive Halving (SH) and its asynchronous extension, ASHA, which provide a simple yet effective framework for allocating resources by periodically promoting the best-performing configurations. While highly efficient, these methods are non-adaptive as they do not use a model of the response surface to guide their decisions.

Bayesian Optimization (BO) has been successfully integrated with multi-fidelity methods to improve sample efficiency Nguyen et al. (2019). BOHB is a prominent example, combining the promotion mechanism of Hyperband (a synchronous version of ASHA) with a Kernel Density Estimator (TPE) to sample new configurations. However, a key limitation of BOHB is that its promotion decisions remain purely heuristic and are not informed by the probabilistic model. In contrast, CAR-HPO uses its surrogate model to drive both sampling and promotion in a unified, principled manner.

More recent methods have focused on making promotion decisions more adaptive. PASHA Bohdal et al. (2022) extends ASHA by using historical rank stability to decide whether a configuration should be promoted. While more adaptive than ASHA, this approach is retrospective, relying on past performance rankings. CAR-HPO adopts a more forward-looking strategy, using its multi-output GP to explicitly predict performance at the maximum budget and making promotion decisions based on this probabilistic forecast.

The work most closely related to ours is DyHPO Wistuba et al. (2022), which represents the state-of-the-art in fully adaptive, model-based multi-fidelity HPO. DyHPO also employs a GP to model learning curves and makes dynamic decisions about resource allocation. The crucial difference lies in the computational strategy. DyHPO refits its model after every single trial completion to make the next decision, leading to extremely high computational overhead. Our core hypothesis is that this per-step refitting is often unnecessary and computationally wasteful. CAR-HPO directly addresses this by introducing episodic, amortized model updates at checkpoints, aiming to achieve similar

decision quality at a fraction of the computational cost. We therefore position CAR-HPO as a direct, practical alternative to DyHPO and include it as a key baseline in our experiments.

Our method’s use of a multi-output Gaussian Process to model correlations between different fidelities is a well-established technique in multi-fidelity BO Li et al. (2020). By explicitly modeling performance at each budget level as a correlated output, the model can effectively transfer information from cheap low-fidelity evaluations to inform predictions at the expensive maximum budget. The challenge, which CAR-HPO addresses, is to leverage this powerful modeling approach within a framework that remains scalable and computationally tractable Wei et al. (2024); Ament & Gomes (2022).

3 BACKGROUND

To understand our proposed method, it is essential to first establish the foundational concepts of multi-fidelity hyperparameter optimization and Bayesian optimization upon which it is built.

3.1 PROBLEM SETTING

The goal of hyperparameter optimization (HPO) is to find a configuration of hyperparameters that minimizes a given objective function, which is typically the validation loss of a machine learning model. Formally, given a hyperparameter configuration space C , we aim to find the optimal configuration c^* that minimizes an expensive-to-evaluate black-box function $f(c)$: $c^* = \operatorname{argmin}_{c \in C} f(c)$.

In the multi-fidelity setting, we assume the objective can be evaluated at varying levels of fidelity, represented by a budget b from a set $B = \{B_{\min}, \dots, B_{\max}\}$. For instance, the budget could be the number of training epochs or the fraction of the training dataset used. A higher budget b corresponds to a more accurate but more costly evaluation $f(c, b)$. The ultimate goal is to find the configuration c^* that minimizes the objective at the maximum budget, $f(c, B_{\max})$, while minimizing the total computational cost incurred during the optimization process.

Multi-fidelity HPO methods based on successive halving operate on a series of ‘rungs’, where each rung corresponds to a specific budget level. A set of configurations is evaluated at the lowest budget. A fraction of these are then ‘promoted’ to the next rung (a higher budget) for further evaluation, and this process is repeated until a few configurations are evaluated at the maximum budget, $B_{\max}$. Asynchronous Successive Halving (ASHA) parallelizes this process, allowing workers to evaluate configurations and request new ones without waiting for all trials on a rung to complete, leading to significant speedups in parallel environments.

3.2 BAYESIAN OPTIMIZATION

Bayesian Optimization (BO) is a sample-efficient strategy for optimizing black-box functions. It operates in two main steps. First, it fits a probabilistic surrogate model, typically a Gaussian Process (GP), to all available observations of ‘(configuration, performance)’ pairs. The GP provides not only a mean prediction for any configuration’s performance but also a measure of uncertainty about that prediction. Second, it uses an acquisition function, such as Expected Improvement (EI) or Lower Confidence Bound (LCB), to balance exploration (evaluating configurations in regions of high uncertainty) and exploitation (evaluating configurations with low predicted mean). The configuration that maximizes the acquisition function is chosen for the next evaluation.

3.3 MULTI-OUTPUT GAUSSIAN PROCESSES

In the multi-fidelity context, we need to model the relationship between performance at different budget levels. A Multi-Output Gaussian Process (GP) is a natural choice for this task. It treats the performance at each budget level (or rung) as a separate but correlated output task. By using an appropriate kernel structure, such as a product of a kernel over the configuration space (e.g., RBF) and an index kernel over the task/budget space, the model can learn the correlation structure between fidelities. This allows observations at low budgets to inform the model’s predictions and uncertainty estimates for performance at high budgets, enabling more efficient and informed decision-making.

4 METHOD

We introduce CAR-HPO (Checkpoint-based Adaptive Racing for HPO), an algorithm that integrates the efficiency of asynchronous successive halving with the intelligence of episodic Bayesian optimization. The core novelty of CAR-HPO is its decision-making process, which is exclusively triggered at adaptive ‘checkpoints’ to amortize the computational cost of its probabilistic model while maintaining high adaptivity.

4.1 ALGORITHM OVERVIEW

CAR-HPO operates as an asynchronous multi-fidelity optimizer, managing a pool of parallel workers. Workers evaluate given ‘(configuration, budget)’ pairs and report back the results. Unlike methods that make a decision after every single result, CAR-HPO collects results and only initiates a full decision-making cycle when a ‘checkpoint’ is reached. A checkpoint for a given promotion rung is triggered after a predefined number of new trial results (N_{new}) have been collected for that rung, or after a maximum time interval has passed. This episodic nature ensures that the model is updated with a substantial batch of new information, making its subsequent decisions more robust and amortizing the fitting cost.

At each checkpoint, a two-phase process executes:

1. **Amortized Surrogate Refitting:** A single, unified multi-output Gaussian Process (GP) is fitted to the entire history of ‘(configuration, budget, performance)’ data collected so far across all rungs.
2. **Unified Model-driven Decisions:** This freshly updated model is immediately used to make two concurrent decisions: promoting existing configurations (exploitation) and sampling new ones (exploration).

4.2 PHASE 1: AMORTIZED SURROGATE REFITTING

The heart of CAR-HPO is its probabilistic surrogate model. We use a multi-output GP that explicitly models the performance at each rung as a distinct, correlated task. This allows the model to capture complex inter-rung dependencies, enabling accurate extrapolation from low-budget to high-budget performance. The GP kernel is structured as a product of two components: a standard kernel for the configuration space (e.g., an RBF kernel for continuous parameters) and an ‘IndexKernel’ for the discrete set of rungs (tasks). This structure allows the model to learn a covariance matrix that captures the similarity between different budget levels. Crucially, this complex model is refitted only at checkpoints, drastically reducing computational overhead compared to the per-step refitting strategy of methods like DyHPO Wistuba et al. (2022).

4.3 PHASE 2: UNIFIED MODEL-DRIVEN DECISIONS

Once the GP model is updated at a checkpoint, it provides a coherent probabilistic belief over the entire search space, which we leverage for both promotion and sampling.

a) Predictive Promotion: For each configuration c that has completed its evaluation at a given rung and is awaiting a promotion decision, we use the GP model to predict its performance distribution at the maximum budget, B_{max} . We identify the current incumbent, $c_{\text{incumbent}}$, as the configuration with the highest posterior mean performance at B_{max} . A configuration c is promoted to the next rung only if its Lower Confidence Bound (LCB) on performance at B_{max} is better than the predicted mean performance of the current incumbent. This criterion, $\text{LCB}(f(c, B_{\text{max}})) > \text{mean}(f(c_{\text{incumbent}}, B_{\text{max}}))$, makes the promotion decision both adaptive and risk-averse. It prioritizes configurations that are not only predicted to be good on average but are also predicted to be good with high confidence. This constitutes a key departure from the heuristic promotions of BOHB and the rank-based promotions of PASHA Bohdal et al. (2022).

b) Adaptive Infill Sampling: Following the promotion decisions, the same GP model is used to generate a new batch of configurations to be evaluated at the lowest budget. We optimize a cost-aware acquisition function, such as Expected Improvement at B_{max} per unit of computational cost, over the configuration space. The optimization yields the most promising configurations to explore

next. The size of this new batch is dynamically adjusted to match the number of currently available workers, ensuring high parallelism and resource utilization. CAR-HPO thus uses a single, coherent probabilistic model to unify and optimize both exploitation (promotions) and exploration (sampling) in a principled and computationally efficient manner.

5 EXPERIMENTAL SETUP

The primary goal of our experiments is to rigorously validate the core hypothesis of CAR-HPO: that it achieves the sample efficiency of high-overhead, fully adaptive methods while maintaining a computational overhead comparable to simpler schedulers. We designed an experiment using tabular benchmarks to isolate and accurately measure scheduler overhead.

5.1 BASELINES AND RATIONALE

We compare CAR-HPO against a spectrum of HPO strategies to provide a comprehensive evaluation:

- **CAR-HPO (Our Method):** Configured with its key episodic mechanism. The checkpoint trigger N_{new} is set to 4, which is half the number of workers, balancing adaptivity with cost amortization.
- **DyHPO Wistuba et al. (2022):** Represents the state-of-the-art in fully dynamic, adaptive HPO. It serves as the high-performance, high-overhead upper bound against which we measure our sample efficiency and computational gains.
- **BOHB:** A widely-used baseline that combines heuristic Hyperband promotions with a TPE model for sampling. This allows us to assess the advantage of CAR-HPO’s unified, model-driven approach for both promotion and sampling.
- **ASHA:** A state-of-the-art, non-model-based scheduler. It is extremely low-overhead and serves as a baseline to quantify the value added by the model-based components of other methods.
- **Random Search (Multi-fidelity):** Implemented as asynchronous random search within a successive halving framework. This establishes the performance floor that any advanced HPO method must convincingly outperform.

5.2 BENCHMARK SELECTION

We use the HPOBench library for reproducible and well-vetted tabular benchmarks. The tabular nature allows for near-instantaneous function evaluations, making the scheduler’s own runtime the primary bottleneck and thus ideal for stress-testing overhead.

- **CartpoleSwingShort_v0:** A benchmark with a low-dimensional, continuous search space to test optimization of continuous parameters.
- **BNNOnProteinStructure_v0:** A more complex benchmark with a higher-dimensional, mixed-type (continuous, integer, categorical) search space to test the robustness of the surrogate models.

5.3 EXPERIMENTAL PARAMETERS

All experiments are run under a unified simulation framework with the following parameters:

- **Wall-Clock Limit:** 3600 seconds (1 hour).
- **Parallelism:** 8 workers ($N_{\text{WORKERS}} = 8$), simulating a realistic multi-core environment.
- **Replications:** 10 replications with different random seeds ($N_{\text{SEEDS}} = 10$) for statistical robustness.

5.4 IMPLEMENTATION DETAILS

The CAR-HPO surrogate model is implemented using `GPyTorch`. We use a `gpytorch.models.MultiTaskGP` to explicitly model the performance at each budget level as a correlated output. The kernel is a product of a `gpytorch.kernels.RBFKernel` for the configuration space and a `gpytorch.kernels.IndexKernel` for the fidelities (tasks), as this structure is central to the CAR-HPO methodology. Data analysis and plotting are performed using `pandas`, `matplotlib`, and `seaborn`.

5.5 METRICS AND ANALYSIS PROTOCOL

- **Primary Metrics:**

1. **Incumbent Trajectory:** The best objective value found so far as a function of wall-clock time, measuring overall performance.
2. **Cumulative Scheduler Overhead:** The total time spent within the optimizer’s internal logic, directly measuring computational cost.

- **Aggregate Metric:** We calculate the Area Under the Curve (AUC) for the log-scaled incumbent trajectory plot. This provides a single scalar metric for overall performance across the entire time budget.
- **Statistical Analysis:** We perform pairwise Wilcoxon signed-rank tests on the final AUC values between CAR-HPO and each baseline to determine if observed performance differences are statistically significant ($p < 0.05$).

6 RESULTS

Our experiments were designed to evaluate CAR-HPO on two key axes: sample efficiency (performance) and computational efficiency (overhead). We compared CAR-HPO against Random Search, ASHA, BOHB, and DyHPO on two HPOBench tabular benchmarks over a one-hour wall-clock time limit, with results averaged over 10 random seeds.

6.1 PERFORMANCE AND SAMPLE EFFICIENCY

The primary performance metric was the Area Under the Curve (AUC) of the log-transformed incumbent trajectory, where lower values indicate better performance. Our results strongly support the hypothesis that CAR-HPO achieves state-of-the-art sample efficiency.

On the `CartpoleSwingShort_v0` benchmark, CAR-HPO and DyHPO were the clear top performers. CAR-HPO achieved a mean AUC of 5.68, which was statistically indistinguishable from DyHPO’s mean AUC of 5.61 (Wilcoxon signed-rank $p = 0.6548$). Both methods demonstrated a statistically significant advantage over BOHB ($p = 0.0020$), ASHA ($p = 0.0010$), and Random Search ($p = 0.0010$), confirming the superiority of their adaptive, model-driven strategies.

On the more complex `BNNOnProteinStructure_v0` benchmark, CAR-HPO emerged as the single best-performing method. It achieved a mean AUC of -2.45, showing a statistically significant improvement over all baselines. Notably, it outperformed even the fully-dynamic DyHPO ($p = 0.0488$), suggesting that its episodic update mechanism might lead to more robust model fits in complex spaces. The performance gap was even larger when compared to BOHB ($p = 0.0020$), ASHA ($p = 0.0010$), and Random Search ($p = 0.0010$).

These results demonstrate that CAR-HPO’s unified, model-driven promotion and sampling strategy provides state-of-the-art sample efficiency, matching or exceeding that of the computationally intensive DyHPO method.

6.2 COMPUTATIONAL OVERHEAD

We explicitly measured the cumulative scheduler overhead, which is the time the optimizer spends on internal computations. The results highlight the core advantage of CAR-HPO’s design.

DyHPO, with its per-step model refitting strategy, incurred an extremely high mean overhead of approximately 1250 seconds. This consumed over 34% of the total one-hour budget, leaving significantly less time for actual function evaluations. In stark contrast, CAR-HPO exhibited a much lower mean overhead of approximately 250 seconds. This is roughly 5 times lower than DyHPO’s overhead and consumed a much smaller fraction of the total budget. CAR-HPO’s overhead was in the same order of magnitude as BOHB’s (~ 180 seconds), while ASHA and Random Search had negligible overhead (~ 25 s and ~ 15 s, respectively), as expected.

This analysis confirms that CAR-HPO’s episodic, amortized model refitting mechanism is highly effective. It successfully avoids the prohibitive computational costs of fully dynamic methods while retaining the powerful modeling capabilities necessary for top-tier performance, thus optimizing the performance-per-computation trade-off.

7 CONCLUSION

In this work, we addressed the critical trade-off between adaptability and computational efficiency in multi-fidelity hyperparameter optimization. We identified a gap between low-overhead but rigid schedulers and highly adaptive but computationally prohibitive fully dynamic methods. To bridge this gap, we introduced CAR-HPO, a novel episodic Bayesian racing algorithm.

Our primary contribution is an amortized intelligence framework where a powerful, unified multi-output Gaussian Process model is refitted only at adaptive checkpoints. This single model is then used to make principled, concurrent decisions for both predictive, risk-averse promotion and adaptive, cost-aware sampling. This design allows CAR-HPO to make robust, model-informed decisions without incurring the crippling overhead of constant model refitting.

Our extensive experiments on tabular benchmarks provide strong evidence for the effectiveness of our approach. The results show that CAR-HPO achieves sample efficiency that is competitive with, and in some cases superior to, the state-of-the-art fully dynamic method DyHPO. Crucially, it achieves this performance with a computational overhead that is five times lower, placing it in a practical range comparable to simpler methods like BOHB. By successfully occupying a novel sweet spot on the performance-versus-computation spectrum, CAR-HPO proves to be a powerful, practical, and efficient HPO solution. Future work could explore the application of CAR-HPO to more complex problems such as neural architecture search, or investigate alternative surrogate models and acquisition functions within the episodic framework. This work democratizes advanced adaptive optimization, making it feasible for researchers and engineers with constrained computational budgets to efficiently tune expensive models.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Sebastian Ament and Carla Gomes. Scalable first-order bayesian optimization via structured automatic differentiation. 2022.
- Ondrej Bohdal, Lukas Balles, Martin Wistuba, Beyza Ermis, Cédric Archambeau, and Giovanni Zappella. Pasha: Efficient hpo and nas with progressive resource allocation. 2022.
- Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research* 20 (2019) 1-21, 2018.
- Shibo Li, Wei Xing, Mike Kirby, and Shandian Zhe. Multi-fidelity bayesian optimization via deep neural networks. 2020.
- Bruno Mlodozieniec, Matthias Reisser, and Christos Louizos. Hyperparameter optimization through neural network partitioning. 2023.
- Vu Nguyen, Sebastian Schulze, and Michael A Osborne. Bayesian optimization for iterative learning. 2019.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.

Yunyue Wei, Vincent Zhuang, Saraswati Soedarmadji, and Yanan Sui. Scalable bayesian optimization via focalized sparse gaussian processes. 2024.

Martin Wistuba, Arlind Kadra, and Josif Grabocka. Supervising the multi-fidelity race of hyperparameter configurations. 2022.